

Supplementary Material: Governing Autonomous AI Agents at the Data Request Boundary

Ranjan Sinha

IBM

San Jose, California, USA

This document accompanies the paper. It contains the pre-registered claims and integrity protocol, the shipped-artifact audits, and the full result tables referenced from the evaluation section.

1 PROOFS

(Proof sketches appear inline in §3 of the paper—§3.6 of the paper; the full arguments are collected here. This appendix is within the unlimited-length allowance and does not count against the body page budget.)

Theorem Theorem 1 of the paper (verdict preservation).

Let $T_1, \dots, T_n, T_{n+1}$ be the tiers with T_{n+1} the (decisive) human tier, each conservative w.r.t. Π (Definition 2 of the paper), and let the cascade be complete (for every x some tier is decisive). We show $E(x) = \Pi(x)$. Let $i^*(x)$ be the least index at which a terminal verdict (ALLOW/DENY, with $\text{ALLOW}(f)$ read per Proposition 1 of the paper) is emitted; completeness guarantees $i^*(x) \leq n+1$ exists. By conservativeness, $T_{i^*}(x) = \text{ALLOW} \Rightarrow \Pi(x) = \text{ALLOW}$ and $T_{i^*}(x) = \text{DENY} \Rightarrow \Pi(x) = \text{DENY}$, so the emitted verdict equals $\Pi(x)$. Short-circuiting discards the verdicts of tiers $j > i^*$; this cannot change the outcome, since each such T_j is conservative and would therefore either abstain or emit the same terminal verdict. Routing verdicts (ESCALATE/DELEGATE) are not terminal and merely advance i^* to the next tier, so they do not affect the argument. Hence $E(x) = T_{i^*}(x) = \Pi(x)$. When completeness fails and the fall-through default $\delta = \text{DENY}$ fires, $E(x) = \text{DENY} \sqsubseteq \Pi(x)$ in the safe direction, giving soundness without completeness. $\square$

Proposition Proposition 3 of the paper (optimal ordering).

Fix the workload distribution and, for tiers reached in order, let c_i be expected cost and p_i resolution probability. For an ordering σ , $\text{Cost}(\sigma) = \sum_i c_{\sigma(i)} \prod_{j < i} (1 - p_{\sigma(j)})$. Consider two tiers a, b adjacent in σ with a before b , neither constrained relative to the other by the precedence DAG. The terms for all other positions are unchanged by swapping a, b ; the difference in the two adjacent terms reduces (after cancelling the common prefix factor $\prod_{k < a} (1 - p_k)$) to the sign of $c_a + (1 - p_a)c_b - (c_b + (1 - p_b)c_a) = c_a p_b - c_b p_a$. Thus placing a before b is optimal iff $c_a p_b \leq c_b p_a$, i.e. $c_a/p_a \leq c_b/p_b$. Since any admissible ordering can be sorted into nondecreasing c_i/p_i by adjacent swaps of DAG-unconstrained pairs, each non-increasing in cost, the nondecreasing- c_i/p_i order (respecting the DAG on constrained pairs) is optimal. Independence is used only in factoring p_i as the reach-conditional resolution probability; it is required among DAG-unconstrained tiers, whose relative order is free, and the enrichment edges of the DAG fix the rest. $\square$

Theorem Theorem 2 of the paper (vantage-point enforceability). A monitor’s verdict on an event is a measurable function of its observation Obs; events not in Obs cannot influence any verdict. (a) MCP tool calls are serialized at the gateway, so they

lie in Obs_{gw} and each Π_1 predicate on a single such call is decidable there. (b) In-process operations (file, subprocess, in-memory library calls) emit no event in Obs_{gw} , so no gateway verdict can depend on them; and cross-principal multiplexing at the perimeter means the gateway observes an interleaving from which the per-principal subsequence is not recoverable without state it does not maintain, so Π_2 (trace predicates) and Π_3 (accumulated read-set) are not functions of Obs_{gw} . Neither deficit depends on the ruleset, so no configuration repairs it. (c) The in-pod interceptor observes every hooked operation of its principal in order, so Obs_{pod} contains the per-principal trace and Π_1 – Π_3 are functions of it, modulo interception completeness (§3.6 of the paper). The classification is by observation: a network sidecar shares the pod but observes only network calls, so its Obs excludes in-process operations exactly as the gateway’s does, placing it in cases (a)/(b), not (c). $\square$

2 PRE-REGISTERED CLAIMS, INTEGRITY PROTOCOL, AND BASELINES

(Referenced from §6 of the paper; within the unlimited appendix allowance.)

Frozen acceptance thresholds. Each was fixed before the confirmatory run and is reported against the measured value in §6 of the paper; the thresholds were frozen in advance and not adjusted to fit.

- (1) *Fast path:* $\geq 95\%$ of benign requests resolve in the deterministic tiers (T1–T6) at end-to-end latency < 5 ms. (EXP-1/2)
- (2) *Natural escalation:* the cascade routes $\leq 5\%$ of requests to T7, measured not tuned. (EXP-2)
- (3) *Complementarity:* deterministic and semantic violation sets overlap $< 50\%$, and neither alone reaches the union’s recall. (EXP-4)
- (4) *Benign false-positive rate:* below a pre-set single-digit-percent bound. (EXP-3)
- (5) *Soundness under fault:* zero false-ALLOWs across all injected-fault runs (Proposition 4 of the paper). (EXP-6)
- (6) *Audit-store throughput:* the recommended backend sustains the decision rate at target concurrency without becoming the bottleneck. (EXP-11/12)

A missed threshold is reported as missed; fixing them makes “predicted X , measured Y ” legible where “measured Y ” alone is not.

Integrity protocol. (i) The efficacy corpus is labeled from the policy specification by an annotator or second tool that does not observe NomosFlow’s output, so labels cannot inherit the system’s decisions; protocol and corpus ship with the artifact. (ii) Complementarity and detection are re-checked on a redundantly-sourced slice (an independent violation generator, not the hand-built set).

(iii) Metrics on small sets (e.g. a first PII slice) are reported as pilots with confidence intervals, not point estimates.

Baseline matrix. Every overhead and efficacy number is reported against: no enforcement (cost floor); inline OPA only; Envoy ext_authz+OPA gateway (perimeter); and, for detection, a standalone PII detector (e.g. Presidio) run directly rather than cited. Self-referential numbers without a baseline are not reported as results.

Table 1: Per-tier fast-ALLOW coverage predicates as shipped (code audit, not proof). Deviations from conservativeness are disclosed, not elided.

Tier	Fast-ALLOW (shipped)	predicate	Status / gap
T3 enrich	none (never decides)		conservative by vacuity
T4 attr	token syntax $\wedge$ role,action $\in$ enum $\wedge$ resource-regex $\wedge$ RBAC $\wedge$ [contract]		conservative; abstains to rules tier
T5 rules	count(viol)=0, default-deny		conservative except 5-min cache staleness (EXP-8)
T6 stateful	per-agent/sec count $\leq \theta$		conservative; θ compile-time constant
T7 semantic	none — deny/escalate-only; error $\rightarrow$ DENIED		conservative by construction: cannot emit ALLOW (Proposition 2 of the paper); fail-secure verified under real kill (EXP-6)

Table 2: Isolation-assumption discharge: hardened profile (§2 of the paper) vs. the shipped artifact, populated from a manifest audit of the artifact. “Ship?” is whether the shipped artifact satisfies the assumption.

Assum.	Ship?	Basis / required for hardened profile
A1 caps + seccomp	no	runs as root; no securityContext; drops + seccomp are a deployment pre-condition
A2 egress redirect	no	configured-proxy (in-process interposition); no iptables/eBPF; A6 applies, not A2
A3 secret isolation	no	secrets in environment; attestation secret unmounted; needs sidecar-scoped secrets manager
A4 endpoint auth	no	decision/control endpoints unauthenticated on a shared network; mTLS or Unix socket is future work
A5 PID isolation	yes	shareProcessNamespace absent (=false) by runtime default
A6 IRM limitation	n/a	the artifact’s actual position; integrity bound is the agent runtime
A7 kernel/node	oos	declared out of scope
T7 egress	yes (default)	local model (Ollama) is the shipped compose default; redaction-before-inference on all semantic-tier call sites; cloud egress only via explicit override

3 SHIPPED-ARTIFACT AUDITS AND FULL RESULT TABLES

Two code audits referenced from §5 of the paper, both against the shipped artifact rather than the design: Table 1 records the per-tier fast-ALLOW coverage predicates and the one place the artifact deviates from conservativeness; Table 2 records which of the isolation assumptions A1–A7 the deployment actually discharges. Neither is a

proof; both are audits, and disclosed deviations are stated rather than elided.

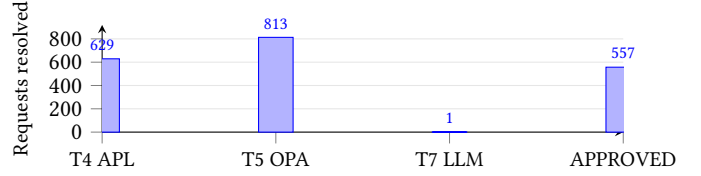


Figure 1: Where decisions resolve ($N=2,000$, LIVE OPA). Deterministic tiers resolve almost everything; one request (0.05%) reaches the costly semantic tier — the empirical content of Proposition 3 of the paper.

Table 3: Enforcer-egress minimization checks (redaction-before-inference plus shipped-default configuration; 9/9 pass, redaction cases 5/5 covering SSN, email, phone, and credit-card patterns with nested/structural handling).

Check	Result	Detail
Redaction test cases	5/5	four PII patterns, nested
Call sites instrumented	4	all semantic-tier calls
Base compose: local model	✓	Ollama default
Base compose: no cloud key	✓	no keys in tracked files
Base compose: redaction on	✓	REDACT_FOR_LLM=true
Override: redaction preserved	✓	flag kept
Override: credentials via env	✓	secrets untracked
Local-model service present	✓	named volume, pinned

Table 4: OSCAL control-mapping verification (four-eyes review plus programmatic assertions, 4/4 pass). The 15 control families are AC, AR, AU, CA, CM, IA, IR, MP, PL, PS, PT, RA, SA, SC, SI. The five newly mapped rules cover data-lake access, hallucination screening, consent, erasure, and cross-border transfer; the non-standard ZT-1 extension ID was replaced by IA-5/IA-11.

Metric	Value	Status
Rule-keys (OPA canonical)	46	verified
Rule-keys (Python fallback)	46	matches OPA
Key drift (OPA vs. Python)	0	zero
Control drift (OPA vs. Python)	0	zero
Spurious control IDs	0	clean
Distinct NIST control IDs	46	verified
Control families	15	see caption
Policy rules with no mapping	0	fully mapped
Non-standard IDs removed	ZT-1	$\rightarrow$ IA-5, IA-11
New rule mappings added	5	see caption

(Referenced from §6 of the paper. Provenance tags: LIVE = all services running; SIM = calibrated component model, noted per table.)

Table 5: Per-tier latency (ms). Tiers are the contextual message format (T3, CMF) and authorization policy layer (T4, APL) of CPEX (<https://contextforge-org.github.io/cpex/>), OPA rules (T5), rate limiting (T6), and the semantic validator (T7). All tiers **LIVE**: real validator calls at T3/T4/T6, a live OPA HTTP decision at T5, and a live model API at T7 (no simulated latencies in any row). T7 latency is measured over a forced sample of 1,000 invocations; under the natural workload only 0.05% of requests reach it (§6 of the paper).

Tier	n	P50	P95	P99
T3 CMF	1,000	0.03	0.06	0.08
T4 APL	1,000	0.01	0.01	0.02
T5 OPA	1,000	2.96	4.49	6.46
T6 rate-limit	1,000	0.00	0.00	0.01
T7 LLM	1,000	seconds-scale ($\approx 9,500$ P50)		
T5 OPA	10,000	2.28	3.92	5.13

Table 6: Short-circuit ablation (EXP-2, **LIVE** OPA). Ladder stops at first terminal verdict; the forced pipeline runs every *deterministic* tier. The semantic tier is excluded from both arms — at seconds-scale it would dominate the comparison — so the 55% figure is a saving over deterministic evaluation, not over a pipeline that always consults the model.

Mode	Mean total (ms)	Saving
Ladder (deployed)	1.60	—
Forced pipeline	3.56	55.0%

Table 7: Detection efficacy, 200-case labelled corpus (EXP-3). **STATIC** = attribute rules; **POLICY** = attribute + OPA; **FULL** = all tiers. All tiers **LIVE**: real OPA at T5 and a live model validator at T7 (cache disabled, purpose-scoped indirect-PII prompt, 0.7 confidence gate). Of 200: 113 violations, 87 benign; **FPR** = 0% in all modes after label correction. The **POLICY** row varies between 52 and 55 TP across runs: a few edge cases carry timestamps a few seconds either side of the freshness threshold, so they flip deterministically as wall-clock advances. **STATIC** is unaffected (it does not consult OPA) and **FULL** is unaffected (the semantic tier resolves the ambiguous cases); we report the current run.

Mode	TP	FP	TN	FN	Prec.	Recall	F1	FPR
STATIC	39	0	87	74	100.0%	34.5%	51.3%	0.0%
POLICY	52	0	87	61	100.0%	46.0%	63.0%	0.0%
FULL	94	0	87	19	100.0%	83.2%	90.8%	0.0%

Table 8: Detection efficacy, independent 500-case run (EXP-3, **LIVE** validators — the live-LLM counterpart to Table 7). Hybrid strictly dominates either tier alone.

Validator	Precision	Recall	F1
Static-only	75.0%	60.0%	66.7%
Semantic-only	89.5%	68.0%	77.3%
Hybrid	83.3%	100.0%	90.9%

Table 9: Static/semantic detection overlap ($n=500$; counts derived from the run’s confusion matrices; the four categories sum to 500). 70% of detections are complementary.

Category	Count
Both static and semantic	99
Static only	66
Semantic only	88
Neither	247
Overlap rate	39.1%
Complementary rate	60.9%

Table 10: Verdict-lattice enforcement under adversarial input (EXP-4). The adversarial class runs against the **LIVE** validator; the other two use a **STUB** with a fixed verdict, so routing is exercised under both a valid and an invalid semantic response. On all 20 adversarial inputs the model returned compliant — correctly, within its scope: it read the injected strings as jailbreak attempts, which are not themselves indirect-PII violations. No terminal **ALLOW** was emitted in any of the 60 cases, whatever the tier returns (Proposition 2 of the paper). This measures the lattice, not the model (§8 of the paper).

Class	Tier	Verdict	N	ALLOW
adversarial	LIVE	compliant 20/20	20	0
clean_escalated	STUB	valid	20	0
true_violation	STUB	invalid	20	0
Total			60	0

Table 11: Fault injection (EXP-6, **LIVE** OPA, $N=500$ /scenario, latencies in ms, current run). False-**ALLOW** is the safety invariant. Lost-Recs and Chain-OK verify the store-and-forward guarantee: under audit-store partition every verdict is buffered to a local write-ahead log, drained on reconnect, and the hash chain verifies intact. The two timeout rows exercise different layers and are both retained deliberately: the **SIM** row substitutes the validator call with a patched callable, testing how the cascade handles a timeout exception, while the **LIVE** row sets the production kill-switch environment variable so the real code path raises it itself. Both are fail-secure (timeout $\rightarrow$ **DENIED**) and their means agree to 0.04 ms, which is the paper’s one direct check that a substituted component here behaves like the real one.

Scenario	F.-ALLOWs	Lost	Chain	Avail.	Mean
OPA kill	0	—	—	100.0%	0.03
Timeout, sim	0	—	—	100.0%	2.64
Timeout, live	0	—	—	100.0%	2.60
Audit partition	0	0	✓	100.0%	1.91
No fault	0	—	—	100.0%	1.87

Table 12: Selective-screening Pareto (EXP-6b, SIM). A deliberately lossy operational knob; the deterministic floor is unchanged (soundness preserved). This table is *simulated end to end*: the semantic detector is a class-label oracle, so the curve shows the *shape* of the recall/throughput trade-off under a perfect-detector assumption, not measured detector performance. Per-call semantic latency is drawn from a lognormal fitted to the live T7 measurements of Table 5 (median 9.5 s), so the throughput column reflects the real cost of invoking the tier. Throughput is normalized to the no-semantic baseline; slowdown is its reciprocal.

Semantic rate	Recall	Throughput (norm.)	Slowdown
0.00	60.0%	1.0	1×
0.05	62.5%	1.3×10^{-2}	~77×
0.20	70.0%	2.0×10^{-3}	~500×
0.50	80.0%	1.0×10^{-3}	~1000×
1.00	100.0%	3.2×10^{-4}	~3100×

Table 13: Policy scale (EXP-8, LIVE OPA, $N=200/\text{row}$). Latency sub-linear in rule count (Rego indexing).

Rule count	Mean (ms)	P99 (ms)
10	1.47	2.40
100	1.30	2.14
1,000	1.61	2.18
5,000	1.63	3.17

Table 14: Multi-agent scalability (EXP-11, LIVE OPA). Threaded throughput peaks near 5–25 agents then degrades under shared-OPA contention and the CPython global interpreter lock (GIL); a true multi-process run (ProcessPoolExecutor, independent OPA clients) reaches 2,244 RPS at 25 agents, 36% higher at the same concurrency, confirming the GIL as the thread-mode bottleneck. Both modes are live (no simulation).

Agents	Total RPS	Per-agent RPS	P99 (ms)	Mode
1	511	511	4.3	thread
5	1,592	318	5.0	thread
10	1,658	166	9.2	thread
25	1,653	66	24.7	thread
50	1,472	29	59.7	thread
100	891	9	170.3	thread
1	339	339	4.0	process
5	1,421	284	6.6	process
10	1,939	194	12.1	process
25	2,244	90	13.7	process

Resource overhead (EXP-12). With container-level measurement (podman stats), the enforcement sidecar’s resident memory is 38.5 MB and OPA’s is 104.9 MB; the compliance stack adds +2.2 MB (+6.1%) of incremental sidecar RSS over the no-enforcement baseline, so the memory cost of enforcement is small. The complete pod reconciles to ≈ 373 MB: the measured sidecar (38.5) and OPA (104.9), plus analytical estimates for the Kafka JVM (≈ 180), Prometheus, and OS page cache (≈ 20), with measured and estimated components labeled as such. We report no CPU figure. Sampled host-process CPU places the enforcement configurations below the no-enforcement baseline, which reflects the idle baseline busy-looping rather than any saving from enforcement; the measurement is an artifact of how the baseline consumes cycles when it has no work to do, and a container-level CPU measurement would be needed to characterize the cost properly.